

RELAZIONE TECNICA

Sistema di Rilevamento Anomalie Industriali con Agentic AI e MCP Server

Industrial IoT Predictive Maintenance Platform

Corso	Scalable and Reliable Services M
Anno Accademico	2025/2026
Data	Maggio 2026
Dominio	Industrial / IoT Monitoring

Indice

1. Descrizione del Servizio.....*Errore. Il segnalibro non è definito.*

2. Architettura del Sistema 6

3. Layer Agentico e MCP Server 9

4. Stack di Kubernetes12

5. Reliability e Scalability.....15

6. Observability e Monitoraggio16

7. Esperimenti di Failure20

8. Costi e Analisi del Ritorno (ROI e ROA)21

9 – Processo di sviluppo e problemi riscontrati.....24

10. Conclusioni e sviluppi futuri25

1. Descrizione del Servizio

1.1 Servizio realizzato

Il progetto sviluppato consiste in una piattaforma di Predictive Maintenance per ambienti industriali, progettata per monitorare in tempo reale lo stato operativo di macchinari industriali, rilevare anomalie e supportare decisioni operative tramite un sistema agentic basato su AI e infrastruttura cloud-native.

Il sistema acquisisce continuamente dati di telemetria provenienti da macchine industriali (in questo esempio, quattro), in particolare relativi a:

- vibrazione,
- temperatura,
- velocità di rotazione (RPM).

Questi parametri rappresentano indicatori fondamentali per valutare lo stato di salute dei macchinari e identificare tempestivamente eventuali condizioni di degrado o funzionamento anomalo.

I dati vengono raccolti tramite un'architettura distribuita basata su MQTT e successivamente gestiti attraverso Apache Kafka, che consente la gestione affidabile e scalabile dei flussi di eventi. Le informazioni vengono elaborate da un modulo di anomaly detection e utilizzate da un sistema decisionale basato su agenti AI per la gestione delle successive azioni operative.

L'obiettivo del sistema è quindi non solo rilevare anomalie, ma anche interpretarle e supportare la gestione operativa degli eventi industriali, mantenendo la continuità produttiva e riducendo il rischio di fermi macchina non pianificati.

1.2 Contesto e motivazioni

Nel contesto industriale moderno, la continuità operativa degli impianti produttivi rappresenta un fattore critico per la competitività aziendale. Un guasto non rilevato tempestivamente può infatti causare il blocco completo di una linea produttiva, con impatti significativi sia economici che operativi.

In particolare, il malfunzionamento di componenti critici come motori, cuscinetti o sistemi di trasmissione può propagarsi rapidamente ad altri sottosistemi, aumentando i tempi, la complessità dell'intervento e i tempi di ripristino.

Secondo stime di settore, il costo di un fermo macchina non pianificato nel settore manifatturiero può variare da decine di migliaia fino a centinaia di migliaia di euro all'ora, includendo perdita di produzione, ritardi nella supply chain e costi di manutenzione straordinaria.

In questo scenario, la manutenzione reattiva risulta inefficiente e costosa, poiché interviene solo dopo il verificarsi del guasto. Per questo motivo si sta diffondendo l'approccio di **manutenzione predittiva**, che consente di identificare in anticipo segnali di degrado e anomalie prima che si verifichi il guasto.

Questo approccio permette di passare da una logica reattiva a una gestione **proattiva e data-driven**, riducendo:

- tempi di inattività,
- costi di manutenzione non pianificata,
- rischio di guasti critici,
- impatto sulla produzione.

L'obiettivo finale è aumentare l'affidabilità degli impianti industriali e migliorare la continuità operativa.

1.3 Obiettivi del sistema

L'obiettivo del sistema è realizzare una piattaforma di manutenzione predittiva intelligente in grado di monitorare in tempo reale lo stato di macchine industriali, rilevare anomalie e supportare la continuità produttiva.

In particolare, il sistema si pone i seguenti obiettivi:

- monitoraggio continuo della telemetria industriale (vibrazione, temperatura, RPM),
- rilevamento di anomalie tramite approccio ibrido,
- classificazione degli eventi (guasti, warning, falsi positivi),
- ottimizzazione della continuità produttiva attraverso ribilanciamento del carico,

- supporto alle decisioni operative tramite automazione controllata,
- garantire scalabilità e affidabilità del sistema.

L'obiettivo finale è la realizzazione di un sistema end-to-end capace di osservare, analizzare e supportare la gestione degli eventi industriali in modo strutturato e affidabile.

1.4 Architettura generale del sistema

Il sistema è stato progettato secondo un'architettura distribuita e cloud-native per garantire scalabilità e separazione dei componenti.

In una prima fase, il sistema è stato sviluppato utilizzando Docker, che ha permesso la containerizzazione dei servizi e una gestione semplificata dell'ambiente di sviluppo. Questa scelta ha facilitato la prototipazione iniziale del sistema, garantendo portabilità e riproducibilità dei componenti software. Successivamente, con l'aumento della complessità del sistema e la necessità di supportare carichi più elevati e scenari più realistici, l'architettura è stata migrata su Kubernetes, introducendo meccanismi avanzati di orchestrazione come autoscaling, self-healing e gestione automatica dei guasti.

I dati di telemetria vengono raccolti tramite protocollo MQTT, utilizzando EMQX come broker adatto a contesti IoT industriali. I flussi di dati vengono successivamente gestiti tramite Apache Kafka, che agisce come sistema di streaming distribuito, buffering degli eventi e disaccoppiamento tra componenti.

La persistenza dei dati è affidata a MongoDB, utilizzato per memorizzare telemetria storica, alert, ticket e log operativi.

Il sistema è orchestrato tramite Kubernetes, che gestisce il deployment dei servizi garantendo scalabilità orizzontale, resilienza ai guasti e continuità operativa anche in caso di failure dei singoli componenti.

La componente di osservabilità è implementata tramite Grafana, che consente il monitoraggio in tempo reale delle metriche del sistema e dello stato delle macchine.

Nel complesso, l'architettura combina tecnologie di containerizzazione, streaming distribuito e orchestrazione cloud-native, costituendo una base robusta su cui si innestano i moduli di anomaly detection e il layer di AI agentici descritti nelle sezioni successive

1.5 Sistema di anomaly detection

Il sistema di anomaly detection rappresenta uno dei componenti centrali della piattaforma e ha l'obiettivo di identificare in tempo reale comportamenti anomali nei macchinari industriali a partire dai dati di telemetria.

L'approccio adottato è di tipo **ibrido**, in quanto combina tecniche basate su regole deterministiche con modelli di Machine Learning. Questa scelta consente di bilanciare reattività, accuratezza e robustezza del rilevamento.

La componente deterministica si basa su soglie operative definite sui principali parametri monitorati (temperatura, vibrazione, RPM), consentendo l'individuazione immediata di condizioni critiche.

Parallelamente, la componente basata su Machine Learning consente invece di rilevare anomalie più complesse e degradazioni progressive non identificabili tramite soglie statiche.

Il sistema produce eventi strutturati che includono:

- tipo di anomalia,
- livello di severità,
- confidence score.

Questi output vengono utilizzati dal livello decisionale del sistema per la gestione operativa.

1.6 Architettura agentic

A valle del sistema di anomaly detection, la piattaforma integra un'architettura basata su agenti AI specializzati per supportare la gestione delle anomalie.

L'approccio agentic consente di separare le responsabilità decisionali in più livelli logici, migliorando la controllabilità del sistema e riducendo il rischio di azioni non coerenti o non sicure. Gli agenti operano su un'interfaccia controllata (MCP Server), che consente l'accesso a funzionalità del sistema in modo regolato e tracciabile.

L'architettura è composta da tre agenti principali:

- un agente dedicato all'analisi e classificazione delle anomalie,

- un agente responsabile dell'ottimizzazione della continuità produttiva,
- un agente incaricato dell'esecuzione delle azioni operative.

Il primo agente si occupa principalmente di interpretare i dati di telemetria e fornire una classificazione del tipo di evento rilevato. Il secondo agente utilizza queste informazioni per valutare l'impatto sulla produzione complessiva e proporre strategie di ribilanciamento. Il terzo agente, infine, traduce le decisioni in azioni operative concrete, come la modifica dei parametri di funzionamento delle macchine o l'apertura di interventi di manutenzione.

Questa separazione consente di disaccoppiare analisi, decisione e azione, migliorando affidabilità e controllo del sistema.

1.7 Scalabilità e affidabilità

Il sistema è progettato per garantire scalabilità e affidabilità in contesti industriali caratterizzati da elevati volumi di dati e requisiti di continuità operativa.

In una fase iniziale, il sistema è stato sviluppato in ambiente Docker, che ha permesso una gestione efficiente dei servizi e una rapida prototipazione.

Successivamente, è stato adottato Kubernetes per introdurre capacità avanzate di orchestrazione e gestione distribuita.

Questa evoluzione ha permesso di introdurre meccanismi fondamentali per la gestione di sistemi distribuiti, tra cui la possibilità di scalare dinamicamente i servizi in base al carico e garantire una maggiore resilienza ai guasti.

In particolare, l'architettura supporta:

- scalabilità orizzontale, tramite la replicazione automatica dei servizi in base alla domanda,
- self-healing, ovvero la capacità del sistema di ripristinare automaticamente componenti non funzionanti,
- gestione dei failover, per garantire continuità operativa anche in caso di guasti parziali dell'infrastruttura,
- alta disponibilità dei servizi, riducendo il rischio di interruzioni del sistema.

Questo consente alla piattaforma di mantenere prestazioni stabili anche in presenza di variazioni del carico o guasti parziali dell'infrastruttura.

2. Architettura del Sistema

2.1 Overview dell'architettura

Il sistema è progettato secondo un'architettura distribuita ed event-driven, pensata per gestire in modo continuo i dati provenienti da macchinari industriali e trasformarli in decisioni operative. L'intero sistema segue una logica a pipeline end-to-end, in cui ogni componente ha un ruolo specifico nella gestione del flusso informativo.

L'architettura è organizzata in **cinque layer funzionali principali**, ciascuno con responsabilità ben definite:

- acquisizione dei dati dai macchinari industriali (edge layer),
- trasporto e raccolta dei dati tramite protocollo MQTT,
- gestione dello streaming tramite message bus distribuito,
- persistenza dei dati storici e operativi,
- elaborazione, anomaly detection e decision making.

A questi si affiancano ulteriori livelli trasversali dedicati all'esecuzione delle azioni e al monitoraggio del sistema.

Questa suddivisione consente di ottenere un sistema modulare, facilmente scalabile e robusto rispetto ai guasti dei singoli componenti. La separazione tra i vari livelli consente inoltre di ridurre le dipendenze dirette tra i servizi, migliorando la robustezza complessiva del sistema.

2.2 IoT Layer – Acquisizione dei dati

Il primo livello del sistema è rappresentato dall'edge layer, che ha il compito di generare e raccogliere i dati di telemetria.

I dati vengono prodotti con una frequenza costante e per testare il sistema sono stati introdotti anche scenari di fault injection, cioè situazioni simulate di guasto o degradazione progressiva.

La comunicazione dei dati avviene tramite protocollo MQTT, utilizzando EMQX come broker. Questa scelta è stata fatta perché MQTT è leggero, efficiente e adatto a contesti industriali e IoT, dove è necessario trasmettere dati frequenti con bassa latenza.

I dati vengono pubblicati su topic strutturati secondo il formato `factory/{machine_id}/sensors`, che permette di identificare in modo chiaro la sorgente di ogni flusso informativo. Il modello publish/subscribe adottato permette di disaccoppiare completamente le macchine dal resto del sistema, rendendo l'architettura più flessibile ed estendibile.

In un contesto industriale reale, le macchine fisiche sarebbero configurate per pubblicare direttamente su questi topic, senza necessità di modifiche architetturali grazie al disaccoppiamento garantito dal protocollo MQTT.

2.3 Data Ingestion Layer – Apache Kafka

Il livello di ingestion rappresenta il cuore dell'architettura event-driven del sistema. Il suo compito principale è quello di garantire una gestione affidabile e scalabile del flusso continuo di dati provenienti dai macchinari industriali.

I dati generati nel layer MQTT vengono trasferiti verso Apache Kafka tramite un bridge dedicato. Kafka viene utilizzato come message bus centrale del sistema e rappresenta il punto di disaccoppiamento tra la fase di acquisizione e quella di elaborazione.

La scelta di Kafka è guidata dalla necessità di gestire flussi di dati ad alta frequenza in tempo reale, mantenendo al tempo stesso affidabilità e scalabilità. In particolare, Kafka consente:

- buffering degli eventi per aumentare la resilienza
- disaccoppiamento tra produttori e consumatori
- persistenza temporanea dei dati per replay e rielaborazioni
- scalabilità orizzontale del throughput

Il sistema MQTT-Kafka bridge ha il compito di tradurre i messaggi tra i due mondi: si sottoscrive ai topic MQTT, deserializza i messaggi in formato JSON e li pubblica nei topic Kafka.

All'interno del sistema, Kafka è organizzato in tre stream principali:

- sensor-readings → dati di telemetria delle macchine
- anomalies → possibili anomalie individuate dal sistema di detection non-AI
- alerts → anomalie, con aggiunta l'analisi degli agenti
- machine-commands → comandi operativi verso il livello di esecuzione

Questa separazione consente di modellare chiaramente i diversi tipi di eventi che attraversano il sistema.

I dati vengono consumati da più componenti in parallelo: il modulo di anomaly detection, il sistema di persistenza su MongoDB, il layer agentico e il layer di execution. Questa architettura multi-consumer garantisce scalabilità e separazione delle responsabilità.

Nel complesso, Kafka costituisce il backbone dell'intera architettura event-driven, collegando il mondo IoT con i livelli di processing, decision making e storage.

2.4 Data Storage Layer – MongoDB

Il livello di storage ha il compito di garantire la persistenza dei dati generati e utilizzati dal sistema. Per questo motivo è stato scelto MongoDB, un database NoSQL particolarmente adatto alla gestione di dati semi-strutturati come quelli provenienti dalla telemetria industriale.

Il database memorizza diverse tipologie di informazioni:

- telemetria storica delle macchine, utilizzata per analisi e modelli ML
- eventi di anomaly detection con relativo livello di severità
- ticket di manutenzione e remediation
- audit log delle decisioni prese dal sistema

MongoDB è stato scelto per la sua flessibilità, scalabilità orizzontale e per la naturale compatibilità con dati in formato JSON, che rappresentano la struttura tipica dei messaggi IoT. Le sue funzionalità di replicazione sono sfruttate per garantire l'affidabilità dei dati in caso di perdita di una delle repliche.

Il database non svolge solo una funzione di storage, ma è parte attiva della pipeline. I dati storici vengono infatti utilizzati sia dal modulo di anomaly detection sia dal layer agentico per contestualizzare le decisioni e migliorare la qualità dell'analisi.

2.5 Processing Layer – Anomaly detection pipeline

Il Processing Layer rappresenta il livello intermedio tra la semplice raccolta dei dati e il sistema decisionale intelligente. Il suo obiettivo è trasformare i dati grezzi di telemetria in informazioni strutturate utilizzabili dai moduli successivi del sistema.

I dati provenienti da Kafka vengono elaborati attraverso una pipeline che include:

- consumo dei messaggi dal topic sensor-readings
- normalizzazione dei dati di telemetria
- aggregazione temporale dei segnali
- arricchimento con informazioni storiche provenienti da MongoDB
- preparazione dei dataset per il modulo di anomaly detection

Il risultato di questa fase non è ancora una decisione, ma un insieme di informazioni strutturate che descrivono lo stato della macchina e che includono eventuali segnali preliminari di anomalia. Questi dati vengono poi utilizzati dal modulo di anomaly detection e dal layer agentico per le fasi successive.

2.6 Observability Layer

Il sistema integra un livello di osservabilità progettato per garantire il monitoraggio continuo dello stato dell'infrastruttura e delle macchine industriali.

Questo aspetto rappresenta un elemento fondamentale in architetture distribuite, poiché consente di comprendere il comportamento interno del sistema attraverso l'analisi di metriche, log ed eventi.

Nel progetto, l'osservabilità è principalmente implementata tramite **Grafana**, utilizzato per la visualizzazione e il monitoraggio in tempo reale delle principali metriche del sistema.

Le principali informazioni monitorate includono:

- stato operativo delle macchine industriali

- valori di telemetria (temperatura, vibrazione, RPM)
- throughput dei flussi Kafka
- latenza dei componenti di elaborazione
- stato dei container e dei servizi Kubernetes
- performance complessive del sistema

Il sistema di osservabilità consente di identificare anomalie infrastrutturali, supportare il debugging e monitorare le prestazioni dei vari componenti, fornendo una visione unificata dell'intero sistema distribuito.

2.7 Infrastructure Layer

L'infrastruttura del sistema segue un approccio cloud-native basato su containerizzazione e orchestrazione. Con l'aumento della complessità del sistema, l'infrastruttura è stata successivamente migrata su Kubernetes. Questa evoluzione ha introdotto funzionalità fondamentali per un sistema distribuito, tra cui:

- scalabilità automatica dei servizi
- gestione del carico tra le istanze
- self-healing in caso di failure
- deployment controllati senza interruzioni
- maggiore resilienza complessiva

Grazie a Kubernetes, il sistema è in grado di mantenere continuità operativa anche in condizioni di carico elevato o in presenza di guasti, rendendo l'intera piattaforma più robusta e adatta a scenari industriali reali.

2.8 Data Flow End-to-End

Il sistema segue una pipeline completamente event-driven che collega in modo continuo la generazione dei dati alla loro elaborazione e alle successive azioni operative.

I dati vengono generati dalle macchine industriali (in questo esempio, simulate), che rappresentano l'edge layer del sistema. Ogni macchina produce continuamente telemetria relativa a vibrazione, temperatura e velocità di rotazione (RPM).

I dati vengono pubblicati tramite protocollo MQTT su topic dedicati e raccolti dal broker EMQX. Successivamente, un bridge MQTT-Kafka si occupa di trasferire i messaggi verso Apache Kafka, che rappresenta il message bus centrale del sistema.

Una volta entrati nel sistema, i dati vengono consumati dai componenti del processing layer e dal sistema di anomaly detection, che analizzano i flussi in tempo reale per identificare eventuali comportamenti anomali.

Quando viene rilevata un'anomalia, il sistema genera eventi strutturati che vengono utilizzati dal layer decisionale basato su agenti AI, per elaborare ed eseguire un piano d'azione che può modificare la configurazione delle macchine o lanciare notifiche ad operatori umani. Parallelamente, i dati vengono persistiti su MongoDB per garantire storicizzazione e tracciabilità.

Infine, l'intero sistema viene monitorato tramite Grafana, che consente la visualizzazione in tempo reale dello stato delle macchine e delle principali metriche operative.

Questo flusso garantisce un'elaborazione continua, scalabile e resiliente dei dati industriali lungo tutta la pipeline.

3. Layer Agentico e MCP Server

3.1 Motivazioni della scelta agantica

La scelta di utilizzare un'architettura agantica nasce dalla necessità di gestire situazioni industriali complesse in modo più flessibile rispetto a un sistema puramente deterministico basato su regole statiche.

In un approccio tradizionale, ogni anomalia genera generalmente una risposta predefinita. Tuttavia, in un ambiente industriale reale, la gestione di un fault richiede valutazioni più articolate: non è sufficiente rilevare che una macchina presenta un problema, ma è necessario capire quanto il guasto sia grave, quale impatto possa avere sulla produzione e quali azioni siano realmente sostenibili dal punto di vista operativo.

Per questo motivo si è scelto di introdurre un layer agentico composto da agenti specializzati, ciascuno con responsabilità ben definite. In questo modo il processo viene suddiviso in più fasi separate:

- analisi tecnica dell'anomalia,
- valutazione dell'impatto produttivo,
- esecuzione delle azioni operative.

Questa separazione permette di rendere il sistema più controllabile e modulare, evitando che un singolo componente abbia contemporaneamente responsabilità di analisi ed esecuzione.

Un ulteriore vantaggio dell'approccio agentic è la capacità di interpretare il contesto operativo in modo più dinamico, consentendo agli agenti di collaborare nella valutazione del rischio e nella definizione di strategie di mitigazione più adatte rispetto a una logica reattiva statica.

Un aspetto fondamentale dell'architettura riguarda la sicurezza operativa. Gli agenti dedicati all'analisi e all'ottimizzazione operano esclusivamente in modalità read-only, mentre le operazioni di modifica del sistema sono affidate a un componente esecutivo separato con privilegi controllati. Questo approccio segue il principio di least privilege e riduce il rischio di azioni non autorizzate o eccessivamente aggressive.

3.2 Architettura del sistema agentic

L'architettura del sistema è basata su tre agenti specializzati, ciascuno responsabile di una specifica fase del processo decisionale. La comunicazione tra gli agenti non avviene in modo diretto, ma attraverso eventi generati dal sistema di anomaly detection e informazioni rese disponibili dal MCP Server.

Il flusso operativo segue una pipeline sequenziale che parte dall'analisi dell'anomalia, prosegue con la valutazione dell'impatto produttivo e si conclude con l'esecuzione delle azioni di remediation. Questa struttura consente di mantenere separata la logica interpretativa da quella esecutiva, riducendo il rischio che decisioni non validate si traducano direttamente in azioni operative.

3.3 Senior Data Reliability Engineer (Analyst)

Il primo agente del sistema è il Senior Data Reliability Engineer, anche indicato come Analyst, responsabile dell'interpretazione dei dati provenienti dal sistema di anomaly detection e dalla telemetria industriale.

L'agente simula il comportamento di uno specialista in analisi vibrazionale, termografia e diagnostica industriale, con l'obiettivo di classificare correttamente gli eventi rilevati.

L'obiettivo principale dell'Analyst è determinare se un evento rappresenti:

- un guasto reale,
- una situazione di warning,
- un falso positivo.

Per svolgere questo compito utilizza esclusivamente capability in modalità read-only esposte dal MCP Server, come la consultazione delle letture recenti, degli alert attivi e delle baseline delle macchine.

Attraverso il confronto tra dati correnti e storico operativo, l'agente produce una classificazione dell'evento includendo informazioni come il tipo di guasto, il livello di severità, un confidence score e un valore di RPM sicuro consigliato.

L'Analyst non dispone di alcuna capacità di modifica del sistema, in modo da garantire che il suo ruolo rimanga limitato esclusivamente alla fase di analisi.

3.4 Production Continuity Manager (Optimizer)

Il secondo agente è il Production Continuity Manager, responsabile della continuità produttiva del sistema. Questo agente riceve in input i risultati dell'Analyst, lo stato complessivo della flotta, gli RPM correnti e la capacità residua delle macchine operative.

Il suo obiettivo è ridurre al minimo l'impatto delle anomalie sulla produzione complessiva, cercando di mantenere stabile il sistema anche in presenza di macchine degradate. Per farlo utilizza informazioni fornite dal MCP Server e applica una logica di ottimizzazione che include il calcolo del deficit produttivo, la valutazione dell'headroom disponibile sulle altre macchine e la redistribuzione del carico produttivo entro limiti di sicurezza.

Quando si verifica un'anomalia, l'Optimizer cerca inizialmente di mantenere la continuità produttiva riducendo gli RPM della macchina problematica e redistribuendo il carico sulle altre macchine sane, aumentandone temporaneamente la velocità entro limiti di sicurezza predefiniti. Nel caso in cui il deficit produttivo superi una soglia definita o non possa essere compensato senza superare i limiti operativi consentiti, il sistema genera automaticamente una raccomandazione di escalation verso un operatore umano.

Il risultato finale dell'agente è un piano operativo che include i nuovi target di RPM, le macchine coinvolte nella redistribuzione e una stima dell'impatto produttivo residuo.

3.5 Predictive Maintenance Expert (Specialist)

Il terzo agente è il Predictive Maintenance Expert, che rappresenta il livello esecutivo del sistema. A differenza degli altri agenti, il suo ruolo non si limita all'analisi o all'ottimizzazione, ma consiste nella traduzione delle decisioni in azioni operative concrete.

L'agente riceve le informazioni prodotte dai livelli precedenti e dispone di capability MCP controllate che gli consentono di eseguire le seguenti operazioni:

- riduzione degli RPM della macchina problematica,
- aggiornamento dei target produttivi,
- apertura automatica di ticket di manutenzione,
- richiesta di intervento umano

I comandi operativi sono registrati all'interno del sistema come operazioni controllate. L'esecuzione è demandata a componenti separati del control layer, consentendo di mantenere disaccoppiata la logica decisionale dal controllo diretto dei macchinari industriali, e di simulare la separazione tra livello cloud e livello IoT/fisico che esiste in contesti industriali.

La presenza di un singolo agente esecutivo separato dagli altri livelli decisionali consente di centralizzare il controllo delle operazioni ad alto impatto e applicare policy di sicurezza più rigorose.

Inoltre, tutte le azioni eseguite vengono registrate nei log operativi e tracciate all'interno del sistema di audit, garantendo completa osservabilità e accountability delle decisioni prese.

3.6 MCP Server

Il sistema agentic interagisce con l'infrastruttura industriale esclusivamente tramite MCP Server, che funge da livello intermedio tra agenti e sistema operativo.

Gli agenti non accedono direttamente a database o servizi, ma operano attraverso capability controllate, suddivise in funzionalità read-only e operative. Le prime permettono la consultazione di telemetria e alert, mentre le seconde consentono modifiche controllate come aggiornamenti dei parametri, apertura di ticket o richieste di escalation.

Ogni richiesta viene validata prima dell'esecuzione e registrata nei log di audit, garantendo la tracciabilità completa delle operazioni e la possibilità di ricostruire il comportamento del sistema.

3.7 Sicurezza operativa e Operational Safety Policy

Poiché il sistema opera in un contesto industriale potenzialmente critico, particolare attenzione è stata dedicata alla sicurezza operativa del layer agentic e al controllo delle azioni automatiche eseguite dal sistema.

Per questo motivo è stata definita una *Operational Safety Policy*, con l'obiettivo di limitare i rischi associati all'automazione e garantire che gli agenti operino sempre entro vincoli controllati e tracciabili.

Uno dei principi fondamentali adottati è il *Principle of Least Privilege*, secondo cui ogni agente dispone esclusivamente delle capability strettamente necessarie al proprio ruolo operativo. In particolare, Analyst e

Optimizer operano in modalità read-only, mentre lo Specialist è l'unico agente con capacità operative, comunque limitate e soggette a validazione.

Questa separazione consente di ridurre il rischio operativo e impedisce che un singolo componente possa contemporaneamente analizzare, decidere ed eseguire azioni senza controlli intermedi.

La policy distingue inoltre tra:

- azioni completamente automatizzate,
- azioni supervisionate,
- azioni che richiedono escalation umana.

In particolare, il sistema attiva escalation nei casi in cui la confidence è bassa, le anomalie non sono chiaramente classificabili, il deficit produttivo è elevato oppure vengono richieste operazioni critiche come shutdown o modifiche oltre i limiti consentiti.

Per evitare remediation speculative o comportamenti non sicuri, il sistema adotta un approccio conservativo: in presenza di informazioni insufficienti o scenari ambigui, gli agenti preferiscono limitare l'azione o richiedere escalation piuttosto che eseguire modifiche aggressive sul sistema produttivo.

Tutte le operazioni vengono inoltre registrate in un audit trail completo, che include agente responsabile, input, decisione e risultato.

Questo approccio garantisce tracciabilità completa delle decisioni e permette di ricostruire il comportamento del sistema in fase di debugging o analisi post-incident.

3.8 Guardrail, validazione e controllo delle allucinazioni

Per garantire affidabilità e robustezza del sistema agentico, sono stati introdotti diversi meccanismi di controllo finalizzati a limitare comportamenti non desiderati, loop decisionali o generazione di azioni non coerenti con il contesto industriale.

Uno dei principali guardrail implementati riguarda la limitazione delle iterazioni degli agenti. Ogni agente può infatti eseguire un numero massimo di cicli *Thought* → *Action* → *Observation*, evitando loop di ragionamento indefiniti e consumo incontrollato di risorse computazionali.

Il sistema implementa inoltre timeout operativi sull'intera crew agentica. Qualora il processo decisionale non venga completato entro una finestra temporale definita, l'esecuzione viene automaticamente interrotta ed effettuata escalation verso un operatore umano.

Ulteriori controlli riguardano le azioni considerate non consentite. In particolare:

- nessun agente può effettuare direttamente lo shutdown di una macchina,
- non è consentito superare i limiti operativi configurati,
- non è possibile bypassare le policy di sicurezza,
- le richieste considerate pericolose vengono automaticamente rifiutate.

Le capability operative implementano inoltre controlli sui parametri accettati, impedendo ad esempio:

- riduzioni degli RPM sotto soglie minime di sicurezza,
- richieste di override dei limiti configurati,
- modifiche incoerenti con lo stato operativo corrente.

In caso di violazione delle policy, la richiesta viene rifiutata, registrata nell'audit trail e convertita automaticamente in richiesta di escalation verso un operatore umano.

Poiché il sistema utilizza agenti AI per supportare decisioni operative, sono stati introdotti anche meccanismi specifici per limitare il rischio di allucinazioni o generazione di azioni non coerenti con il contesto industriale.

Le richieste vengono validate su due livelli: uno strutturale, che verifica la correttezza formale delle tool call, e uno semantico, che analizza il contenuto per identificare azioni potenzialmente pericolose. In caso di violazione, la richiesta viene rifiutata e registrata, con eventuale escalation verso un operatore umano.

Questi meccanismi permettono di ridurre il rischio di allucinazioni operative e garantire che il sistema agisca sempre entro vincoli controllati e verificabili.

4. Stack di Kubernetes

L'architettura si basa su un cluster Kubernetes ospitato su Oracle Cloud Infrastructure (OCI), composto da due nodi di calcolo basati su processori ARM64 (Compute Ampere A1), ciascuno dotato di 2 OCPU e 12 GB di RAM, con 100 GB di storage. Il cluster è accessibile dall'esterno tramite un Network Load Balancer (NLB) e un ingress-nginx che ha anche la funzione di reverse-proxy e si articola su una rete virtuale (VCN) con subnet pubblica e privata. L'intera infrastruttura è definita e gestita tramite Terraform, secondo il paradigma Infrastructure as Code.

All'interno del cluster, i carichi di lavoro applicativi vengono distribuiti e gestiti attraverso manifest Kubernetes in formato YAML, con il supporto di Helm per la gestione dei pacchetti e di operatori specializzati — Percona Operator per il MongoDB e Strimzi per Apache Kafka — che automatizzano la gestione di componenti stateful complessi. Nei paragrafi che seguono vengono presentati in dettaglio ciascuno di questi strumenti, con una descrizione del loro funzionamento e dei vantaggi che offrono.

4.1 Terraform: Infrastructure as Code

Terraform è uno strumento open source che consente di definire, pianificare e gestire infrastruttura IT tramite file di configurazione dichiarativi, secondo il paradigma noto come **Infrastructure as Code** (IaC). Aniché configurare manualmente server, reti e altri componenti attraverso interfacce grafiche o comandi eseguiti a mano, con Terraform l'intera infrastruttura viene descritta in codice e applicata in modo automatico e ripetibile.

I principali vantaggi di questo approccio sono:

- **Riproducibilità:** Applicando gli stessi file terraform è possibile ottenere la stessa identica infrastruttura risultate in qualsiasi momento.
- **Idempotenza:** Terraform confronta lo stato desiderato con quello attuale e applica solo le modifiche necessarie, garantendo consistenza senza effetti collaterali indesiderati o operazioni superflue.
- **Automazione e velocità:** l'intero stack infrastrutturale — rete, nodi di calcolo, bilanciatori del carico, storage — viene creato o aggiornato con un singolo comando, eliminando operazioni manuali ripetitive e soggette a errori.
- **Multi-cloud:** lo stesso strumento supporta decine di provider (Oracle Cloud, AWS, Azure, GCP, ecc.), semplificando la migrazione dell'infrastruttura tra cloud diversi.
- **Version control:** essendo la configurazione definita su file di codice veri e propri è versionabile tramite Git, che permette di tracciare le modifiche e migliora la collaborazione tra membri.

Nel nostro contesto, Terraform gestisce la creazione del cluster Kubernetes su Oracle Cloud, inclusa tutta la parte di networking (vcn, subnet, gateway, nat, security policies), il control plane di kubernetes (OKE), e i nodi di calcolo veri e propri.

La configurazione è inoltre fortemente personalizzabile: ad esempio, cambiando un parametro è possibile aumentare o diminuire i nodi in base alle esigenze, e in pochi minuti, essi vengono resi disponibili e aggiunti al cluster Kubernetes.

4.2 Oracle Cloud: vantaggi rispetto a Kubernetes tradizionale

Eseguire Kubernetes su un cloud provider come Oracle Cloud Infrastructure (OCI) offre vantaggi significativi rispetto a un'installazione self-hosted su hardware fisico o macchine virtuali gestite autonomamente:

- **Scalabilità elastica:** è possibile aggiungere o rimuovere nodi al cluster in pochi minuti, adattando le risorse al carico effettivo senza acquistare hardware in anticipo.
- **Alta disponibilità gestita:** il control plane di Kubernetes è completamente gestito dal provider, che ne garantisce la disponibilità e gli aggiornamenti senza richiedere interventi manuali.
- **Integrazione nativa con altri servizi cloud:** il Network Load Balancer per connessioni in entrata è gestito dal provider; lo storage persistente può essere affidato al cloud provider, inclusi i backup periodici.
- **Riduzione del carico operativo:** la gestione della rete fisica, della sicurezza hardware e dei backup di base è delegata al provider, permettendo al team di concentrarsi sulle applicazioni.

Perché Oracle?

Oracle Cloud offre istanze di calcolo Ampere A1 con un rapporto prezzo/prestazioni molto competitivo essendo basate su architettura Arm64 ad alta efficienza energetica.

Il control plane Kubernetes (chiamato OKE – Oracle Kubernetes Engine) è disponibile gratuitamente senza costi aggiuntivi per cluster “basic”, così come anche i load balancer di livello 4 (NLB).

Questa scelta iniziale comunque non preclude la possibilità di migrazione su un provider diverso in un secondo momento.

4.3 Kubernetes: vantaggi rispetto a Docker Compose

Docker Compose è uno strumento efficace per orchestrare più container su una singola macchina in ambienti di sviluppo o per applicazioni di piccole dimensioni. Kubernetes, invece, è una piattaforma di orchestrazione distribuita progettata per ambienti di produzione su larga scala.

Le differenze principali a favore di Kubernetes sono:

- **Alta disponibilità e self-healing:** Kubernetes monitora continuamente lo stato dei container (Pod) e li riavvia automaticamente in caso di crash, distribuendoli su nodi diversi per evitare single point of failure. Docker Compose non dispone di questa capacità in modo nativo.
- **Scalabilità orizzontale:** è possibile scalare un servizio aumentando il numero di repliche con un semplice comando o in modo automatico tramite l'Horizontal Pod Autoscaler, in risposta al carico CPU o a metriche personalizzate.
- **Distribuzione su più nodi:** Docker Compose opera su un singolo host, mentre Kubernetes distribuisce i carichi di lavoro su un cluster di macchine, aumentando resilienza e capacità complessiva.
- **Deployment avanzati:** Kubernetes supporta nativamente strategie come il rolling update (aggiornamento progressivo senza downtime) e il rollback automatico in caso di errore, fondamentali in produzione.
- **Gestione dichiarativa e riconciliazione:** lo stato desiderato del sistema è definito in manifest YAML e Kubernetes si occupa di mantenerlo attivo nel tempo, gestendo automaticamente fallimenti e derive di configurazione.
- **Ecosistema e integrazioni:** Kubernetes dispone di un ricco ecosistema di strumenti (Helm, Ingress controller, service mesh, operatori) che consentono di gestire applicazioni complesse in modo strutturato.

4.4 Deployment

Manifest in formato YAML

Il metodo nativo per descrivere e distribuire risorse su Kubernetes è la definizione di manifest in formato YAML.

Ogni risorsa del cluster — Deployment, Service, Secret, PersistentVolumeClaim, ecc. — viene descritta in un file strutturato che specifica lo stato desiderato dell'oggetto. Kubernetes legge questi manifest e si occupa di creare o aggiornare le risorse corrispondenti, mantenendole allineate alla configurazione voluta nel tempo. Questo approccio è intrinsecamente dichiarativo: non si descrivono i passi da eseguire, ma il risultato che si vuole ottenere.

Nel nostro progetto, i manifest YAML vengono utilizzati per definire i Deployment dei componenti applicativi custom sviluppati e creati interamente da noi.

Helm: gestore di pacchetti per Kubernetes

Helm è il package manager ufficiale per Kubernetes: consente di installare, aggiornare e gestire applicazioni complesse all'interno di un cluster Kubernetes attraverso pacchetti preconfezionati chiamati chart. Un chart Helm è un insieme di template YAML parametrizzabili che descrivono tutte le risorse necessarie per eseguire un'applicazione (Deployment, Service, ecc.).

I principali vantaggi di Helm sono:

- **Riutilizzo e standardizzazione:** esistono chart open source per migliaia di applicazioni (database, proxy, monitoring stack, ecc.), distribuiti tramite repository come Artifact Hub. Questo evita di scrivere da zero manifest complessi e permette di adottare configurazioni già testate dalla community.
- **Parametrizzazione:** i valori configurabili (numero di repliche, risorse CPU/RAM, hostname, credenziali, ecc.) sono isolati in un file values.yaml, rendendo semplice adattare lo stesso chart ad ambienti diversi (sviluppo, staging, produzione) senza duplicare il codice.
- **Gestione del ciclo di vita:** Helm tiene traccia delle release installate nel cluster e supporta nativamente aggiornamenti (helm upgrade) e rollback (helm rollback) a versioni precedenti, con un registro storico delle modifiche.
- **Composizione:** i chart possono includere dipendenze da altri chart (subchart), consentendo di modellare stack applicativi complessi come un'unica unità coerente.

Nel nostro progetto, Helm viene utilizzato per installare tutti i componenti open source disponibili sul web (come MQTT, Kafka, MongoDB, Headlamp, ecc.).

4.4 Operatori Kubernetes

Un operatore Kubernetes è un'estensione del cluster che automatizza la gestione di applicazioni complesse e stateful che richiedono una logica operativa specifica per essere installate, configurate, scalate e mantenute correttamente nel tempo. Gli operatori si basano sul meccanismo delle Custom Resource Definition (CRD): introducono nuovi tipi di risorse Kubernetes comprensibili dal cluster, che l'utente può dichiarare in YAML come qualsiasi altra risorsa nativa.

L'operatore osserva continuamente lo stato del cluster e interviene automaticamente per portarlo allo stato desiderato, applicando la stessa logica che eseguirebbe un amministratore esperto.

Nel nostro progetto utilizziamo gli operatori per la gestione dei due componenti stateful più complessi: MongoDB e Kafka.

MongoDB: Percona operator

Percona Operator è un operatore open source che automatizza la gestione di database all'interno di Kubernetes. Consente di dichiarare un intero cluster database — numero di nodi, dimensione dello storage, politiche di backup, utenti e credenziali — attraverso una singola risorsa YAML.

I vantaggi principali sono: **Alta disponibilità** automatica (replicazione, failover automatico e riconfigurazione); **Backup e restore gestiti** (possibile programmare backup automatici, e di effettuare restore se necessario); **Aggiornamenti** senza downtime (rolling update).

Apache Kafka: Strimzi operator

Strimzi è un operatore open source che porta Apache Kafka all'interno di Kubernetes, semplificandone drasticamente l'installazione e la gestione. Con vantaggi simili a quelli definiti in precedenza per Percona, ma applicati al sistema Kafka. Semplifica drasticamente la gestione di un cluster replicato kafka, che altrimenti necessita della coordinazione e configurazione di più componenti tra loro.

5. Reliability e Scalability

5.1 Gestione della Reliability

Cloud Provider

Essendo l'applicazione interamente ospitata su Oracle Cloud, la gestione della reliability lato hardware è affidata al provider, che si occupa di garantire elevati livelli di disponibilità. Possibilità di fare affidamento su più regioni diverse per avere garanzie ancora maggiori.

Il cloud provider si occupa anche di gestire la reliability del control plane Kubernetes e del load balancer in ingresso, due tra i componenti più critici dell'intero sistema.

Kubernetes

Kubernetes implementa un ciclo continuo di **riconciliazione**: il control plane osserva costantemente lo stato reale del cluster e lo confronta con lo stato desiderato definito nei manifest. Se un Pod termina inaspettatamente, viene riavviato automaticamente; se un intero nodo diventa irraggiungibile, i carichi di lavoro che ospitava vengono ripianificati sugli altri nodi disponibili. Questo comportamento di *self-healing* è trasparente per le applicazioni e non richiede alcun intervento manuale.

Per i componenti stateful, la reliability è affidata agli operatori specializzati (Percona e Strimzi), che attuano politiche ad-hoc per garantire la disponibilità e l'affidabilità del database e del sistema di messaggi.

5.2 Scalabilità

Scalabilità Hardware

A livello di infrastruttura, Oracle Cloud consente di aggiungere nuovi nodi al cluster Kubernetes in pochi minuti, semplicemente modificando la **configurazione Terraform** e applicando le modifiche. I nuovi nodi vengono riconosciuti automaticamente dal cluster e resi disponibili per ospitare i carichi di lavoro, senza necessità di riconfigurare manualmente alcun componente. Questo modello elimina la necessità di sovradimensionare l'hardware in anticipo per far fronte a picchi di carico previsti: le risorse vengono aggiunte quando servono e possono essere ridotte nei periodi di minor utilizzo, ottimizzando i costi operativi.

Scalabilità Software

A livello di applicazione, Kubernetes consente di scalare orizzontalmente qualsiasi servizio aumentando il numero di repliche del Pod corrispondente. Questo può avvenire manualmente, modificando il manifest YAML, oppure in modo automatico tramite l'**Horizontal Pod Autoscaler** (HPA), che monitora metriche come l'utilizzo di CPU e memoria e aggiusta il numero di repliche in tempo reale per mantenere le prestazioni entro soglie definite. È inoltre possibile configurare il **Vertical Pod Autoscaler** (VPA) per ottimizzare automaticamente le risorse assegnate ai singoli container, evitando sia l'over-provisioning che la scarsità di risorse.

Anche i **componenti stateful** — solitamente più complessi da gestire — beneficiano di questa scalabilità: Percona Operator permette di aggiungere nodi al cluster database dichiarativamente, gestendo in autonomia la sincronizzazione dei dati tra le repliche; Strimzi consente di aumentare il numero di broker Kafka e di riassegnare le partizioni dei topic in modo automatico e non distruttivo. In entrambi i casi, la scalabilità avviene senza downtime e senza procedure manuali complesse.

6. Observability e Monitoraggio

6.1 Strategia di Observability dell'Architettura

In un'architettura complessa ed altamente distribuita come quella implementata — basata sullo streaming asincrono tramite Apache Kafka, la persistenza su un cluster MongoDB Sharded e un layer decisionale governato da agenti AI autonomi (CrewAI) — il monitoraggio tradizionale basato unicamente sul controllo manuale dei log dei singoli container risulta del tutto insufficiente. L'operatore umano deve poter essere in grado di ricostruire la catena causale degli eventi in tempo reale. Per fare ciò, la piattaforma implementa una strategia di *Observability* integrata e strutturata su tre livelli logici:

1. Livello fisico: monitoraggio real-time della telemetria grezza proveniente dalle macchine sul campo (velocità di rotazione, vibrazioni, temperature).
2. Livello Diagnostico: visibilità immediata sul livello di criticità rilevato dal modulo di *Fast Detection* e sui flussi di ragionamento sviluppati dagli agenti autonomi prima dell'emissione di un verdetto.
3. Livello esecutivo: tracciabilità in tempo reale delle azioni correttive e dei comandi di riduzione o aumento della velocità impartiti ai simulatori di impianto.

Questa filosofia viene implementata tramite cruscotti operativi ad alto livello (Grafana) ed interfacce di ispezione infrastrutturale a basso livello (Kafka UI e Mongo Express), garantendo al contempo la *compliance* e l'auditing tramite log immutabili strutturati.

6.2 Monitoraggio Operativo High-Level: Grafana Control Center

Grafana è un servizio rappresenta la "torre di controllo" unificata dell'intero sistema. Tramite un plugin dedicato, Grafana si collega direttamente al database di produzione MongoDB; così facendo ha la capacità di effettuare query sui database, ed astrarre la complessità dei documenti JSON grezzi per visualizzare metriche aggregate, indicatori di stato e trend temporali. Grafana permette di gestire e configurare liberamente le proprie dashboard ed i propri pannelli. In seguito si discutono i pannelli più significativi.

Pannelli di telemetria e storico (Gauges e Time Series)

Il monitoraggio visivo della telemetria delle quattro macchine industriali avviene tramite pannelli grafici accoppiati che mostrano lo stato istantaneo e quello storico:

- **Indicatori di velocità istantanea (Gauge):** Ciascuna delle 4 macchine è associata a un pannello di tipo *Gauge* (tachimetro analogico) puntato sul campo rpm della collezione *sensor_readings*. Al fine di emulare fedelmente un cruscotto industriale, il pannello è impostato con soglie cromatiche fisse personalizzate, con una barra di riempimento dinamica che permette all'operatore di percepire a colpo d'occhio la distanza di sicurezza dalle soglie di guasto strutturale del cuscinetto.



- **Trend storico (Time Series):** Le metriche istantanee non permettono di visualizzare lenti fenomeni di degrado progressivo, come l'usura del cuscinetto. A questo scopo viene integrato un pannello *Time Series* che interroga storicamente la collezione `sensor_readings`, filtrando per lo specifico `machine_id` della macchina.
Poiché MongoDB tratta nativamente i campi come testo, è necessario applicare le trasformazioni `Convert field type` di Grafana per poter elaborare correttamente i dati di `rpm` e `timestamp`. Inoltre, per allineare lo sfasamento fuso orario dovuto alla persistenza ISO, la timezone della dashboard viene forzata esplicitamente su UTC, quella utilizzata da MongoDB.



Pannelli di monitoraggio decisionale (Machine Commands ed Escalations Table)

La dashboard presenta inoltre alcuni pannelli di tipo *Table* che interrogano le collezioni `maintenance_tickets`, `alerts`, `machine_commands` ed `escalations`. Questi pannelli visualizzano in tempo reale le decisioni autonome del layer agentic, mostrando l'esito dei comandi inviati dal server MCP di remediation. Attraverso l'uso delle trasformazioni di organizzazione dei campi (`Organize fields`), è possibile mettere in risalto i motivi e le scelte prese dagli agenti, nascondendo informazioni poco utili ad un operatore come l'identificativo interno di MongoDB. L'operatore può così verificare la perfetta correlazione causale: se il grafico *Time Series* della macchina mostra un picco anomalo di vibrazioni, la tabella mostrerà la riga del comando emesso dall'agente (es. riduzione forzata della velocità a 1500 RPM per preservare l'asset).

maintenance_tickets					
machine_id	description	urgency	created_by	status	created_at
machine_c	Diagnosi di FALSO_POSITIVO sulla macchina machine_c. Rilevato picco di vibrazione transitorio e	low	remediation-mcp/agent	open	2026-05-21T14:04:08.525785+00:00
machine_d	Rilevati picchi di vibrazione intermittenti su machine_d, compatibili con uno stadio iniziale di usur	high	remediation-mcp/agent	open	2026-05-21T14:04:12.320430+00:00
machine_a	Rilevata usura cuscinetto con possibile disallineamento (confidence 90%) sulla base di temperat	critical	remediation-mcp/agent	open	2026-05-21T14:04:19.662881+00:00
machine_c	L>alert è stato classificato come FALSO_POSITIVO. L'analisi ha confermato che la macchina opera	low	remediation-mcp/agent	open	2026-05-21T14:07:13.014051+00:00

alerts					
machine_id	ai_reasoning	alert_type	crew_status	severity	timestamp ↓
machine_b	Riepilogo azioni eseguite: - command_id per set_machine_speed_limit: '61ba6812-b20b-47fc-af0e-ba	vibration_x_high	ok	warning	2026-05-23T08:49:51.990864+00
machine_a	Riepilogo azioni eseguite: - command_id per set_machine_speed_limit: '1b5797f9-e44d-475f-abbd-d1	ml_anomaly	ok	warning	2026-05-23T08:47:38.659124+00:
machine_c	Riepilogo azioni eseguite: - **set_machine_speed_limit (machine_c)**: 'command_id: 691ac88f-cf26-4	ml_anomaly	ok	warning	2026-05-23T08:47:36.042571+00:

commands							
machine_id	type	reason	rpm_limit	nominal_rpm	target_rpm	issued_at	
machine_d	speed_limit	Limite ridotto per operare in regime statisticamente più sicuro (anomalia ML), come da report diagnostico.	2300	1850		2026-05-21 17:5	
machine_d	production_target	Compensazione perdita machine_b		1850	2271	2026-05-21 17:5	
machine_c	production_target	Compensazione perdita machine_b		1800	2287	2026-05-21 17:5	
machine_b	speed_limit	Limite ridotto per WARNING: sospetto guasto al sistema di acquisizione dati	700	1750		2026-05-21 17:5	
machine_c	production_target	Compensazione perdita machine_b (+387.26 RPM)		1800	2287	2026-05-21 17:5	
machine_a	production_target	Compensazione perdita machine_b (+191.66 RPM)		1800	1942	2026-05-21 17:5	
machine_b	speed_limit	Limite ridotto per sospetto guasto al sistema di acquisizione dati (WARNING)	700	1750		2026-05-21 17:5	

escalations			
machine_id	reason	status	created_at
machine_b	Escalation richiesta: Il piano di ribilanciamento non è eseguibile. Il comando per machine_b	pending_human_review	2026-05-21T14:32:41.821158+00:00
machine_c	Impossibile eseguire il piano di ribilanciamento. Il piano richiede di impostare la macchina s	pending_human_review	2026-05-21T14:38:06.542621+00:00

6.3 Ispezione Infrastrutturale Low-Level

Mentre Grafana risponde alle necessità quotidiane dell'operatore di impianto, gli ingegneri di sistema e gli sviluppatori necessitano di strumenti a basso livello per ispezionare il corretto funzionamento dei flussi dati intermedi, come i messaggi "in volo" dentro Kafka e lo stato memorizzato in MongoDB.

Kafka UI: Ispezione dei Messaggi in Volo

L'interfaccia grafica Kafka UI consente il monitoraggio in tempo reale del Message Bus distribuito, permettendo di verificare la salute ed il bilanciamento dei singoli topic:

- **sensor-readings:** consente di verificare che il bridge MQTT-Kafka stia correttamente immettendo i flussi di telemetria (8 messaggi/secondo globali) con il corretto partizionamento per chiave (*machine_id*).
- **anomalies:** permette il debug immediato del modulo di *Fast Detection*, mostrando gli snapshot catturati al superamento delle soglie deterministiche o tramite Isolation Forest, prima del coinvolgimento della Crew AI.
- **alerts:** mostra i messaggi JSON di output emessi da CrewAI contenenti l'intero testo di *AI reasoning* e la diagnosi approfondita.
- **scada-events:** traccia l'attività finale di notifica delle variazioni di stato dei simulatori.

Mongo Express: Ispezione dei Dati a Riposo

Mongo Express funge da console web per l'esplorazione visiva del cluster sharded MongoDB. Questo tool è cruciale per ispezionare i documenti complessi scritti dal componente mongo-writer. In particolare, consente di analizzare nel dettaglio le seguenti collezioni:

- **maintenance_tickets:** raccolta dei ticket aperti dall'Agente 2 (*Predictive Maintenance Expert*), verificando il livello di urgenza (low, medium, high, critical) e la coerenza testuale della descrizione tecnica.
- **escalations:** ispezione dei ticket di intervento umano forzato, fondamentali sia quando l'IA rileva la necessità di uno shutdown (azione vietata dalle policy) sia in caso di intervento dei guardrail temporali.
- **alerts:** ispezione dello storico dei log diagnostici dell'IA, utili per analizzare la bontà dei prompt e verificare l'assenza di allucinazioni semantiche.

Headlamp: Controllo e Monitoraggio del Cluster Kubernetes

A completamento della visibilità low-level sull'infrastruttura, viene integrato Headlamp, un'interfaccia grafica per la gestione e l'ispezione del cluster Kubernetes. Mentre Kafka UI e Mongo Express si concentrano sui dati, Headlamp offre una panoramica in tempo reale sullo stato del motore di orchestrazione. Attraverso questo cruscotto, gli sviluppatori possono monitorare lo stato di salute dei Pod, verificare il corretto deployment delle repliche dei servizi applicativi (come il ml-detector o la crew-agent) e accertarsi che i volumi persistenti siano correttamente associati (stato Bound).

Inoltre, la possibilità di visualizzare le metriche di consumo e modificare le risorse dei container direttamente tramite interfaccia grafica semplifica il tuning dei parametri di cluster (CPU/Memory requests e limits), accelerando le sessioni di debug infrastrutturale non dovendo dipendere esclusivamente dalla riga di comando di kubectl.

6.4 Tracciabilità e Compliance: Audit Trail e SCADA Logs

In contesti industriali critici, la sicurezza non può basarsi unicamente sulla fiducia nell'output del modello linguistico (LLM). L'observability deve garantire la massima auditabilità e responsabilità (*accountability*) delle azioni eseguite dall'agente intelligente.

MCP Audit Trail (Layer di Sicurezza Remediation)

Come dettagliato nei meccanismi di sicurezza, il server remediation-mcp implementa la funzione `_policy_check()` di verifica delle policy che analizza ed intercetta keyword vietate come shutdown, power_off o tentativi di violazione dei limiti minimi di sicurezza.

Ogni singola interazione tra la Crew AI e i server MCP viene salvata in modo immutabile in un log strutturato JSON (Audit Trail) dedicato. Ciascun record registra in modo definitivo il timestamp, il server chiamato, il tool specifico invocato, gli argomenti esatti passati dall'agente e l'esito della validazione.

Un tipico esempio di log di audit per una chiamata andata a buon fine si presenta nel seguente formato:

```
AUDIT {"ts": "2026-05-11T07:31:20+00:00", "server": "remediation-mcp",  
      "tool": "set_machine_speed_limit",  
      "args": {"machine_id": "machine_a", "rpm_limit": 1500.0}, "outcome": "ok"}
```

Questo registro garantisce che qualsiasi azione intrapresa dall'agente sia pienamente tracciabile e verificabile a posteriori da auditor umani, fornendo una prova inconfutabile delle richieste espresse dal modello. L'implementazione rigorosa dell'Audit Trail non risponde unicamente a necessità di debugging tecnico, ma rappresenta un requisito fondamentale per la *compliance* legale del sistema nel contesto normativo europeo.

Log dello SCADA Executor

L'anello finale della catena di observability è rappresentato dai log generati dal componente scada-executor. Questo script asincrono monitora la collezione `machine_commands` in stato pending, valida la temporizzazione industriale ed effettua la scrittura fisica (la modifica dei file di configurazione nel prototipo simulato) per alterare i parametri della macchina.

Il log dello SCADA Executor chiude formalmente il cerchio del monitoraggio dimostrando la ricezione del comando digitale e la sua effettiva attuazione sul simulatore, segnalando i dettagli dell'operazione e i tempi di esecuzione schedulati:

```
[SCHEDULATO] machine_a | speed_limit | esecuzione tra 10s  
[ESEGUITO]   machine_a | speed_limit | Riduzione velocita a 1500 RPM  
[ESEGUITO]   machine_b | production_target | Aumento velocita a 1843 RPM
```

Grazie a questo livello finale di tracciabilità, l'operatore può verificare nei log che ad ogni decisione presa dall'IA corrisponda una reale e corretta variazione fisica dell'impianto, garantendo la massima trasparenza e sicurezza operativa dell'intera infrastruttura predittiva.

7. Esperimenti di Failure

7.1 Failure del Broker MQTT

Setup: Evict di un broker MQTT dal cluster Kubernetes.

Comportamento osservato: I simulatori tentano riconnessione automatica ogni 3 secondi (retry loop). Il bridge MQTT-Kafka perde la connessione e tenta riconnessione. Kafka mantiene il consumer group attivo. Nessun dato viene perso su Kafka (I sensor_readings in transito e non ancora trasmessi a kafka vengono persi). Il comportamento normale riprende quasi istantaneamente grazie alla replica del broker che sostituisce in tempo reale quello eliminato.

Recovery: Automatico. Kubernetes riavvia il pod tornando al numero di repliche impostato nel manifest.

7.2 Failure del Broker Kafka

Setup: Evict di un broker Kafka dal cluster Kubernetes

Comportamento osservato: Tutte le partizioni del broker eliminato vengono riassegnate agli altri disponibili. Dopo pochi secondi, Kafka torna operativo, conservando i dati.

Recovery: Automatico. Kubernetes riavvia il pod eliminato. Tramite l'operator Strimzi, Kafka allinea il nuovo pod con le repliche esistenti, e in seguito ribilancia la distribuzione delle partizioni per tornare a operatività normale.

7.3 Failure del Primary MongoDB

Setup: Evict di una replica di MongoDB

Comportamento osservato: Il replica set elegge un nuovo "primary" tra le repliche disponibili. Durante questo periodo (pochi secondi), le scritture del mongo-writer falliscono temporaneamente e vengono messe in retry. Nessun dato è perso.

Recovery: Automatico. Kubernetes riavvia il pod eliminato. Tramite l'operator Percona, i dati vengono allineati alle altre repliche rimanenti. Una volta operativo, il pod viene impostato come "secondary" sul replica set.

7.4 Failure del Servizio LLM

Setup: Configurazione di un OPENAI_API_BASE non raggiungibile, o trovare una call andata in timeout sul topic Kafka.

Comportamento osservato: Fast detection continua normalmente. Quando viene lanciata una crew, LiteLLM fallisce. Il blocco try/except in run_crew_with_guardrails cattura l'eccezione e pubblica l>alert con crew_status=error. Il sistema non si blocca.

Graceful degradation: Il rilevamento base (soglie + ML) rimane operativo al 100%. Solo l'analisi approfondita AI viene persa, che in questo esempio controlla le macchine.

Recovery: Dipendente dal fornitore di LLM. Per questo fallire, ci concentriamo sulla mitigazione.

7.5 Test operatore malevolo

Setup: Tentativo di forzare l'agente a richiedere uno shutdown descrivendo la situazione come emergenziale.

Comportamento osservato: Se il task dell'agente include il testo 'shutdown' o 'emergency_stop', _policy_check() intercetta la keyword e il tool call viene rifiutato con status: rejected prima di qualsiasi scrittura su MongoDB. Inoltre, non è disponibile una funzione MCP che può causare lo shutdown del sistema. Nel caso un'agente tenti di abbassare l'RPM di una macchina sotto a 300, questa richiesta viene rifiutata e viene lanciata un'escalation.

7.5 Failure catastrofico

Setup: Distruzione dell'intero cluster

Comportamento osservato: L'intero sistema non è più disponibile

Recovery: terraform apply (ricrea l'infrastruttura); caricamento dei servizi su kubernetes tramite helm e manifest yaml, e caricamento dei backup per i dati persistenti.

RTO stimato: 3 ore.

8. Costi e Analisi del Ritorno (ROI e ROA)

Questo capitolo analizza le scelte ingegneristiche in relazione al loro impatto economico, dimostrando come l'architettura garantisca affidabilità sotto vincoli di budget e come l'adozione di capacità agentiche (AI) generi un valore operativo misurabile rispetto al suo costo.

8.1 Costo Stimato di Downtime

In un contesto industriale, la failure di una macchina rotante critica come quelle simulate dal sistema comporta il blocco a cascata della linea di produzione. Secondo le stime di settore, il costo del *downtime* non pianificato varia dai 36.000 €/ora per beni di consumo a basso margine, fino a oltre 2.000.000 €/ora in settori come l'automotive.

In questo scenario, la metrica chiave del sistema predittivo è il rilevamento tempestivo, che consente di convertire un guasto emergenziale in un intervento di manutenzione programmata.

L'introduzione dell'agentic AI può migliorare significativamente la manutenzione di un sistema. Un [articolo](#) di IBM stima che l'uso di AI nella manutenzione preventiva può ridurre il downtime fino al 45%, e i costi di manutenzione fino al 30%.

8.2 Service Level Objectives (SLOs)

Per garantire che il sistema risponda alle esigenze operative, sono stati definiti i seguenti SLO:

Metrica SLO	Target	Razionale e Meccanismo di Supporto
Disponibilità Fast Detection	99.9%	Il rilevamento base (soglie deterministiche) non dipende dall'LLM. Se le API AI sono offline, il sistema degrada in sicurezza inviando comunque gli alert.
Recovery Time (RTO) Database	< 60s	Garantito dall'elezione automatica (Raft) in caso di caduta del nodo Primary nel Replica Set di MongoDB.
Latenza di Ragionamento Agentico (CrewAI)	< 120s	Intervento bloccato dal guardrail del <i>Timeout Timeout</i> (SIGALRM) per evitare consumo indefinito di risorse e ritardi di azione.
Alert Loss Rate (Data Loss)	< 0.1%	L'uso di Apache Kafka e il pattern "At-Least-Once" del mongo-writer prevengono la perdita dei messaggi in caso di crash.

8.3 OPEX Mensile e Investimenti in Resilienza

L'infrastruttura è progettata per operare interamente su architetture cloud moderne, come Oracle Cloud OCI tramite OKE (Oracle Kubernetes Engine). Grazie alla containerizzazione e al ridotto footprint dei microservizi, i costi operativi mensili (OPEX) sono altamente ottimizzati:

- **Infrastruttura Cloud (OKE):** ~200€ / mese. Sfruttando nodi worker scalabili (es. AMD E4.Flex) e volumi block storage bilanciati. Il Control Plane di Kubernetes su OCI è gratuito nel tier base.
- **Costo API LLM (Agent Ceiling):** Un'esecuzione della Crew costa circa 0.05€, utilizzando Gemini 2.5 Pro via LiteLLM. Ipotizzando uno scenario conservativo di 2 alert complessi all'ora, il costo si attesta intorno a **75€ / mese**. Il *cost ceiling* è implicitamente imposto dalla deduplicazione degli alert: gli agenti vengono attivati solo al superamento netto della soglia, non su ogni campione inviato a 2Hz.
- **Manutenzione e Monitoraggio:** ~300€ / mese (quota parte del tempo di un ingegnere SRE / DevOps).
- **OPEX Mensile Totale:** ~575€ per il monitoraggio continuo di 4 macchine critiche.

Un Investimento Giustificato in Resilienza:

Per raggiungere l'RTO desiderato (<60s) sui dati operativi, si è scelto di investire in un cluster MongoDB Sharded con Replica Set (3 nodi per shard) anziché usare una singola istanza standalone. Sebbene questo triplichi lo storage richiesto e aggiunga un modesto overhead computazionale, previene la paralisi totale del layer agentico e la perdita dello storico in caso di crash del server o corruzione di un pod.

8.4 Scelte di Design e Trade-Off

È stato necessario effettuare delle scelte di design per alcuni aspetti, con la consapevolezza dei punti a favore e a sfavore di ogni scelta:

- **Trade-off MongoDB Sharded vs Single Instance:** Per raggiungere l'RTO desiderato sui dati operativi (<60s), si è scelto di investire in un cluster MongoDB Sharded con Replica Set (3 nodi per shard) anziché usare una singola istanza standalone. Sebbene questo triplichi lo storage richiesto e

aggiunga un modesto overhead computazionale, previene la paralisi totale del layer agentico e la perdita dello storico in caso di crash del server o corruzione di un pod.

- **Trade-off Costo vs Affidabilità (Always-on vs On-Demand):** Si è scelto di attivare l'agente IA on-demand (solo dopo che il Fast Detector ha rilevato un'anomalia) anziché alimentare uno streaming continuo di dati all'LLM. Questo riduce il costo dei token API di oltre il 99%, barattando una capacità diagnostica teorica costante in favore di una spesa operativa sostenibile, mantenendo comunque un'affidabilità totale nei momenti di reale necessità.
- **Trade-off Automazione vs Sicurezza (Bounded Remediation):** Permettere agli agenti di inviare direttamente comandi di "Emergency Stop" allo SCADA ridurrebbe i tempi di reazione a latenze sub-secondo. Tuttavia, per motivi di sicurezza, la policy impone che comandi distruttivi generino un'*escalation* umana. Il trade-off scelto penalizza l'automazione estrema per garantire che le macchine non vengano bloccate erroneamente da allucinazioni dell'IA, proteggendo la linea produttiva. Questo non impedisce comunque l'approvazione automatica per azioni reversibili a basso impatto, come la leggera modifica della velocità.

8.5 Return on Agent (ROA) e Analisi ROI

L'introduzione di un layer agentico basato su CrewAI e Model Context Protocol (MCP) aggiunge inevitabilmente complessità architetturale (gestione dei server MCP dedicati) ed elementi di rischio (latenza intrinseca degli LLM e potenziale allucinazione nei comandi). Tuttavia, tale incremento è ampiamente giustificato da un valore operativo misurabile che una telemetria deterministica non potrebbe mai offrire. Per quantificare il ritorno economico dell'investimento (ROI) e l'efficacia specifica dell'agente (ROA), l'analisi viene strutturata secondo i requisiti operativi richiesti:

Beneficio Misurabile fornito ai Clienti

In un sistema deterministico, il problema principale non è la mancata generazione di alert, ma l'infestazione da falsi positivi. Un sensore difettoso o una vibrazione transitoria dovuta a un carico momentaneo portano un sistema deterministico a lanciare continui allarmi. In un impianto medio, il 60-80% degli alert tradizionali sono falsi positivi, il cui costo di gestione può comprendere anche l'ispezione visiva da parte di un tecnico, con eventualmente un fermo macchina precauzionale. Questo fa raggiungere costi che oscillano tra i 500€ e i 2.000€ a evento.

L'agente intelligente opera come un *filtro semantico*. Sfruttando i tool del server telemetry-mcp, analizza lo storico recente e le baseline della flotta per verificare la plausibilità fisica del guasto. La Crew è in grado di ridurre il tasso di falsi positivi del **60%**, identificando anomalie strumentali con un livello di confidence del 95%. Su una media di 100 alert mensili generati, l'IA evita circa 60 interventi non necessari, traducendosi in un risparmio diretto stimato tra i 30.000€ e i 120.000€ al mese per il cliente.

(fonti: <https://oxmaint.com/industries/power-plant/power-plant-maintenance-false-alarm-reduction-ai>, <https://www.meritdata-tech.com/resources-post/predictive-maintenance-with-anomaly-detection>).

Modalità di Ottenimento del Ritorno

Il ritorno economico dell'investimento viene catturato trasformando la piattaforma in un servizio Software as a Service (SaaS) ospitato in cloud. Il modello di business prevede un abbonamento mensile ricorrente basato sul numero di macchine monitorate:

- Canale di monetizzazione diretto: Il cliente paga per la garanzia del rispetto degli SLO (ad esempio disponibilità del monitoraggio al 99.9%) e per l'accesso alla dashboard di Grafana.
- Manutenzione su condizione: Il ritorno per il cliente si concretizza nel passaggio dalla costosa manutenzione a calendario (interventi eseguiti anche su componenti sani) alla manutenzione su reale stato di degrado, ottimizzando i cicli di vita dei cuscinetti e riducendo i tempi di diagnostica sul campo del 40-60% grazie al report precompilato dall'IA.

Stima dei Costi di Capitale (CAPEX) e Operativi (OPEX)

Il bilancio economico complessivo del servizio su scala annuale prevede la distinzione tra gli investimenti iniziali e le spese correnti di mantenimento. Le stime sono basate su un impianto pilota di quattro macchine.

Categoria	Voce di Costo	Stima	Dettagli e Razionale
CAPEX	Sviluppo Software Iniziale	~100.000 €	Costo aziendale di un team core (3 ingegneri x 6 mesi) per sviluppo microservizi, con aggiunta di server MCP e orchestrazione CrewAI. Questa categoria è quella che risente di più l'introduzione dell'agentico AI.
CAPEX	Integrazione SCADA/Edge	~30.000 €	Interfacciamento del broker MQTT e dello SCADA executor con i PLC e i macchinari fisici del cliente.

CAPEX	Formazione Personale	~5.000 €	Sessioni di training per gli operatori su Grafana, Headlamp e gestione delle escalation umane.
	TOTALE CAPEX	~135.000 €	<i>Investimento Iniziale. L'uso dell'agentic AI comporta un aumento totale stimato fino a 100.000 €.</i>
OPEX	Infrastruttura Cloud (OCI)	~200 € / mese	Istanze computazionali (OKE), storage a blocchi persistente e 3 nodi dello Sharded Cluster MongoDB.
OPEX	API LLM (Agent Ceiling)	~75 € / mese	Consumo token Gemini 2.5 Pro via LiteLLM per il volume di alert reali (filtrati a monte).
OPEX	Personale / Manutenzione	~500 € / mese	Allocazione di 0.25 FTE di un DevOps/SRE per monitoraggio log e gestione del cluster.
OPEX	Marketing e Vendite	~300 € / mese	Budget per acquisizione clienti B2B nel settore manifatturiero (SaaS growth).
	TOTALE OPEX	~1.075 € / mese	<i>Spesa Operativa Corrente</i>

Ammortizzando l'investimento iniziale CAPEX su di 5 anni (~27.000 €/anno) e aggiungendo l'OPEX annuo operativo (~12.900 €/anno), il costo totale stimato per il mantenimento del servizio si attesta a circa **39.900 €/anno**.

Come derivato dall'analisi dei benefici, la Crew AI riduce del 60% l'impatto dei falsi positivi. Assumendo la stima prudenziale minima e conservativa di 500 € per singolo intervento tecnico non necessario, l'abbattimento dei soli falsi positivi genera un risparmio diretto per il cliente pari a 30.000 €/mese, ovvero **360.000 €/anno** (senza contare l'immenso valore economico derivante dalla prevenzione dei guasti catastrofici reali e del relativo downtime di linea). Questo risulta inoltre in un payback period inferiore ai 6 mesi.

L'integrazione dell'architettura agentica e del Model Context Protocol rappresenta quindi un moltiplicatore di efficienza industriale ed economica ampiamente giustificato: l'overhead economico introdotto dalle API dei modelli linguistici (~900 €/anno) e l'aumento economico dei costi dello sviluppo software sono infinitamente compensati dall'efficienza operativa e dal risparmio sui costi di manutenzione.

9 – Processo di sviluppo e problemi riscontrati

Lo sviluppo del progetto è iniziato su Docker, con un deployment in Docker Compose. L'obiettivo era di garantire isolamento tra le varie parti del progetto e favorire modifiche future. Questo metodo ci ha consentito di aggiungere nuove funzionalità e rimpiazzare tecnologie con relativa semplicità. Una serie di ostacoli sono stati riscontrati durante lo sviluppo, con i più notevoli descritti qui per completezza.

9.1 Docker Compose

Inizialmente, abbiamo lavorato al progetto usando Docker Compose in locale. Questo ci ha semplificato fortemente il deployment di nuovi container per riempire l'infrastruttura, ma si è rivelato un problema perché non c'era modo di testare la scalabilità e resilienza del sistema. Una volta definita e creata la struttura del progetto, abbiamo trasferito tutto su un deployment di Kubernetes, appoggiandoci ad Oracle Cloud per il nostro esempio. L'applicazione include ancora un file docker-compose.yml che ne consente il deployment locale per scopi di testing.

9.2 MQTT e EMQX

Inizialmente, i container simulatori usavano Broker MQTT Mosquitto, simulando l'output di molte macchine industriali. Durante il trasferimento su Kubernetes, abbiamo riscontrato problemi di alta latenza (oltre 3s) e continue disconnessioni del Broker. Questo è stato risolto sostituendo Mosquitto con EMQX, un sistema più robusto e scalabile adatto alle nostre necessità.

9.3 Agenti

L'input degli agenti è stato revisionato più volte, modificandone la backstory e le task per evitare output malformati e favorire l'uso delle funzioni MCP corrette. Abbiamo riscontrato che chiedere agli agenti un output in un formato specifico, adatto alle funzioni MCP utilizzate, fornisce output più consistenti. In ogni caso, l'inconsistenza degli agenti è inevitabile, dovuta ad allucinazioni o semplici problemi di rete. Siamo convinti che un'applicazione reale richiederebbe vasti sistemi di sicurezza e failsafe, ben più estesi di quelli presenti qui.

10. Conclusioni e sviluppi futuri

Abbiamo dimostrato come tecnologie di AI agentic possono essere applicate al campo della fault detection industriale, creando una proof of concept scalabile, resiliente, e fortemente aperta a sviluppi futuri. Inoltre, abbiamo riconosciuto una nicchia di mercato in crescita, che siamo in grado di attaccare con un prodotto accessibile e altamente espandibile in caso di successo, grazie alla flessibilità del modello SaaS basato su servizi di cloud.

Per rendere il nostro progetto market-ready, sono necessarie una serie di estensioni e sviluppi.

10.1 Compatibilità IoT

Vari strumenti nell'applicazione esistono esclusivamente per simulare una struttura reale per scopi dimostrativi e di testing:

- I container "simulatori" in uso nel progetto corrente
- Il broker MQTT che essi usano per comunicare con Kafka
- Lo script scada-executor che applica le modifiche alla velocità dei simulatori

Questi saranno rimossi, e al loro posto sarà installato uno strumento di controllo IoT studiato per la compatibilità con macchinari di più tipi, ad esempio AWS IoT o EdgeXFactory.

10.2 Hardware

L'hardware stesso su cui la nostra dimostrazione esegue è parte di una prova gratuita di Oracle. Per espandere il servizio, sarà necessario investire in una soluzione completa. Il nostro piano consiste nell'estendere l'implementazione con un servizio di cloud, pagando compute e memoria necessari on-demand. Il principale vantaggio di legare la nostra app ad un cloud provider è l'estrema scalabilità delle risorse, che possono essere pagate in base all'uso reale, ed estese con molta facilità.

10.3 Immagini ed efficienza

Per risparmiare memoria attiva e tempo di reazione in seguito ad outage, miriamo a ridurre la dimensione il numero di immagini base usate per i container. Al momento, le immagini compilate occupano più gigabyte di memoria e il deployment di ognuna richiede circa 5-10 secondi. Sarebbe possibile ottimizzare questi valori in due modi:

- Ridurre il numero di container usati, dando ad ognuno più ruoli da svolgere, e quindi risparmiando in costi di esecuzione dell'infrastruttura del container.
- Usare immagini base più leggere e performanti, tagliando funzionalità non necessarie.

10.4 Miglioramenti Agenti

Il sistema di agenti è fortemente limitato dal nostro API, che deve essere esteso e rafforzato. La latenza corrente è stata reputata accettabile per la dimostrazione, ma in un'applicazione reale miriamo a ridurre il tempo di risposta degli agenti per reagire ad emergenze in maniera tempestiva. Sono necessarie ottimizzazioni del processo generativo e di comunicazione fra vari strumenti nell'architettura, e investimenti per un maggior numero di token di generazione.